

Final Database Assignment Title

by

Software Warriors

1.0 Introduction and Background

Educational institutions face significant challenges in coordinating the various components of academic life, including class schedules, assignment deadlines, student study plans, and communication between students and teachers. Manual management of these elements often leads to scheduling conflicts, missed deadlines, and poor coordination across the school community. The need for an integrated system that can efficiently manage these interconnected aspects of academic administration has become increasingly critical in modern educational environments.

Business Scenario

A school scheduling platform is being developed to help schools manage study plans, deadlines, and class schedules. The system will store information about students, teachers, classes and available time slots, while providing an interface to schedule, update and track events. The platform aims to reduce scheduling conflicts, improve planning of future activities and ensure better coordination across the school community.

Entities

Student, Reminder, Study Plan, Class Schedule, Deadline, Teacher, Class

2.0 Business Rules

- One STUDENT attends Many CLASSES, [and possibly/not none]
- One CLASS is attended by Many STUDENTS, [and possibly none]
- One TEACHER teaches Many CLASSES, [and possibly/not none]
- One CLASS is taught by One TEACHER, [and not none]
- One STUDENT has Many REMINDERS, [and possibly none]
- One REMINDER reminds One STUDENT, [and not none]
- One STUDENT has Many STUDY PLANS, [and possibly none]
- One STUDY PLAN belongs to One STUDENT, [and not none]
- One CLASS has One CLASS SCHEDULE, [and not none]
- One CLASS SCHEDULE belongs to One CLASS, [and not none]
- One CLASS has Many DEADLINES, [and possibly none]
- One DEADLINE belongs to One CLASS, [and not none]

- One REMINDER alerts to One DEADLINE, [and not none]
- One DEADLINE has Many REMINDERS, [and possibly none]
- One REMINDER alerts to One STUDY PLAN, [and not none]
- One STUDY PLAN has Many REMINDERS, [and possibly/not none]
- One REMINDER alerts to One CLASS SCHEDULE, [and not none]
- One CLASS SCHEDULE has Many REMINDERS, [and possibly none]
- One TEACHER has Many REMINDERS, [and possibly none]
- One REMINDER reminds One TEACHER, [and not none]

Design of the Database

Student – Class:

Student and Class have a many-to-many relationship, therefore a linking table, called Enrolment, is created. The linking table (Enrolment) includes the foreign keys StudentID and ClassID. There are no constraints, a student or a class can exist without an entry in this table.

Teacher – Class:

Teacher and Class have a one-to-many relationship, therefore a foreign key, TeacherID, is added to the Class table. It is mandatory that TeacherID is in the Class table.

Student – Reminder:

Student and Reminder have a one-to-many relationship, therefore a foreign key, StudentID, is added to the Reminder table. A student may exist without reminders however, a reminder must be linked to a student.

Student – Study Plan:

Student and Study Plan have a one-to-many relationship, therefore a foreign key, StudentID, is added to the Study Plan table. A student may exist without a study plan however, a study plan must be linked to a student.

Class – Class Schedule:

Class and Class Schedule have a one-to-one relationship, therefore a foreign key, ClassID, could be placed in the Class Schedule table or ScheduleID could be placed in the Class table. Every class must be linked to a schedule and vice versa.

Class – Deadline:

Class and Deadline have a one-to-many relationship, therefore the foreign key, ClassID, is added to the Deadline table. A class can exist without deadlines but a deadline must be linked to a class.

Reminder – Deadline:

Reminder and Deadline have a one-to-one relationship, therefore a foreign key, ReminderID, could be placed in the Deadline table or DeadlineID could be placed in the Reminder table. Every reminder must be linked to a deadline and vice versa.

Reminder – Study Plan:

Reminder and Study Plan have a one-to-many relationship, resulting in the foreign key, StudyPlanID, being added to the Reminder table. There is no constraint for study plan to have a reminder but, every reminder must link to a study plan.

Reminder – Class Schedule:

Reminder and Class Schedule have a one-to-many relationship, therefore the foreign key, ScheduleID, is placed in the Reminder table. Schedules can exist without reminders but reminders are linked to schedules.

Teacher – Reminder:

Teacher and Reminder have a one-to-many relationship, therefore the foreign key, TeacherID, is represented in the Reminder table. Reminders are linked to teachers however, a teacher may exist without reminders.

3.0 Entity–Relationship Design

4.0 Relational Schema

relational schema mapped from the ERD

STUDENT (StudentID, Stu_FName, Stu_LName, Stu_Email)

TEACHER (TeacherID, Teach_FName, Teach_LName, Teach_Email, Department)

CLASS (ClassID, TeacherID, Class_Name)

STUDY PLAN (StudyPlanID, StudentID, Subject_Name, Duration, Status)

DEADLINE (DeadlineID, TeacherID, ClassID, Title, Due_Date, Status)

REMINDER (ReminderID, TeacherID, StudentID, Message, Reminder_Time, Notification)

CLASS SCHEDULE (ScheduleID, TeacherID, ClassID, Class_Start_Time, Class_End_Time, Class_Days, Classroom)

ENROLMENT (StudentID, ClassID)

normalization steps

Table Name: Class Details

CLASS DETAILS (ClassID, Class_Name, TeacherID, Teach_FName, Teach_LName, Teach_Email, Department, StudentID, Stu_FName, Stu_LName, Stu_Email)

First Normal Form (1NF)

- No repeating groups in CLASS DETAILS
- Primary Keys Identified: ClassID, TeacherID, StudentID
- Dependencies Identified:

ClassID, TeacherID, StudentID $\rightarrow$ Class_Name, Teach_FName, Teach_LName, Teach_Email, Department, Stu_FName, Stu_LName, Stu_Email

Partial Dependencies:

- ClassID $\rightarrow$ Class_Name, TeacherID, Teach_FName, Teach_LName, Teach_Email, Department
- StudentID $\rightarrow$, Stu_FName, Stu_LName, Stu_Email

Transitive Dependency:

- TeacherID $\rightarrow$ Teach_FName, Teach_LName, Teach_Email, Department

Second Normal Form (2NF)

- Make new tables to eliminate partial dependencies:

CLASS (ClassID, Class_Name, TeacherID, Teach_FName, Teach_LName, Teach_Email, Department)

STUDENT (StudentID, Stu_FName, Stu_LName, Stu_Email)

ENROLMENT (ClassID, StudentID)

Third Normal Form (3NF)

- Make new tables to eliminate transitive dependencies:

CLASS (ClassID, Class_Name)

TEACHER (TeacherID, Teach_FName, Teach_LName, Teach_Email,
Department)

STUDENT (StudentID, Stu_FName, Stu_LName, Stu_Email)

ENROLMENT (ClassID, StudentID)

5.0 Database Implementation

Creating the database to store the tables:

```
Create Schema Database_assignment;
```

```
use database_assignment;
```

Creating the relevant tables for assignment based on ERD

Student Table:

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY, -- student number  
    Stud_FName VARCHAR(50) NOT NULL,  
    Stud_LName VARCHAR(50) NOT NULL,  
    Stud_Email VARCHAR(100) UNIQUE NOT NULL  
);
```

Insert dummy data student:

```
INSERT INTO Student (StudentID, Stud_FName, Stud_LName, Stud_Email)  
VALUES  
(1001, 'John', 'Doe', 'john.doe@school.edu'),  
(1002, 'Alice', 'Brown', 'alice.brown@school.edu'),
```

```
(1003, 'Michael', 'Smith', 'michael.smith@school.edu'),
(1004, 'Sophia', 'Johnson', 'sophia.johnson@school.edu'),
(1005, 'Liam', 'Williams', 'liam.williams@school.edu');
```

StudentID	Stud_FName	Stud_LName	Stud_Email
1001	John	Doe	john.doe@school.edu
1002	Alice	Brown	alice.brown@school.edu
1003	Michael	Smith	michael.smith@school.edu
1004	Sophia	Johnson	sophia.johnson@school.edu
1005	Liam	Williams	liam.williams@school.edu

Teacher Table:

```
CREATE TABLE Teacher (
    TeacherID INT PRIMARY KEY, -- Teacher number/id
    Teach_FName VARCHAR(50) NOT NULL,
    Teach_LName VARCHAR(50) NOT NULL,
    Teach_Email VARCHAR(100) UNIQUE NOT NULL,
    Department VARCHAR(100)
);
```

insert dummy data Teacher:

```
INSERT INTO Teacher (TeacherID, Teach_FName, Teach_LName, Teach_Email,
Department)
VALUES
(2001, 'Emily', 'Taylor', 'emily.taylor@school.edu', 'Mathematics'),
(2002, 'James', 'Anderson', 'james.anderson@school.edu', 'Computer Science'),
(2003, 'Olivia', 'Martin', 'olivia.martin@school.edu', 'English'),
(2004, 'Ethan', 'Harris', 'ethan.harris@school.edu', 'Science'),
(2005, 'Ava', 'Clark', 'ava.clark@school.edu', 'History');
```

TeacherID	Teach_FName	Teach_LName	Teach_Email	Department
2001	Emily	Taylor	emily.taylor@school.edu	Mathematics
2002	James	Anderson	james.anderson@school.edu	Computer Science
2003	Olivia	Martin	olivia.martin@school.edu	English
2004	Ethan	Harris	ethan.harris@school.edu	Science
2005	Ava	Clark	ava.clark@school.edu	History

Class Table:

```
CREATE TABLE Class (
    ClassID INT AUTO_INCREMENT PRIMARY KEY,
    Class_Name VARCHAR(100) NOT NULL,
    TeacherID INT NOT NULL,
    FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID)
    ON UPDATE CASCADE
    ON DELETE CASCADE
);
```

Insert dummy data: Class:

```
INSERT INTO Class (Class_Name, TeacherID)
VALUES
('Algebra 101', 2001),
('Intro to Programming', 2002),
('English Literature', 2003),
('Physics Fundamentals', 2004),
('World History', 2005);
```

ClassID	Class_Name	TeacherID
1	Algebra 101	2001
2	Intro to Programming	2002
3	English Literature	2003
4	Physics Fundamentals	2004
5	World History	2005

Enrollment Table(Bridge Entity):

```
CREATE TABLE Enrollment (  
    StudentID INT NOT NULL,  
    ClassID INT NOT NULL,  
    PRIMARY KEY (StudentID, ClassID),  
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)  
    ON UPDATE CASCADE  
    ON DELETE CASCADE,  
    FOREIGN KEY (ClassID) REFERENCES Class(ClassID)  
    ON UPDATE CASCADE  
    ON DELETE CASCADE  
);
```

insert dummy data enrollment table:

```
INSERT INTO Enrollment (StudentID, ClassID)  
VALUES  
(1001, 1),  
(1001, 2),  
(1002, 1),  
(1002, 3),  
(1003, 2),  
(1003, 4),  
(1004, 3),  
(1004, 5),  
(1005, 4),  
(1005, 5);
```


StudentID	ClassID
1001	1
1002	1
1001	2
1003	2
1002	3
1004	3
1003	4
1005	4
1004	5
1005	5

Deadline Table:

```
CREATE TABLE Deadline (
    DeadlineID INT AUTO_INCREMENT PRIMARY KEY,
    ClassID INT NOT NULL,
    Title VARCHAR(150) NOT NULL,
    Due_Date DATE NOT NULL,
    Status VARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (ClassID) REFERENCES Class(ClassID)
    ON UPDATE CASCADE
    ON DELETE CASCADE
);
```

insert dummy data. Deadline Table:

```
INSERT INTO Deadline (ClassID, Title, Due_Date, Status)
VALUES
(1, 'Algebra Test 1', '2025-10-20', 'Pending'),
(1, 'Homework 3', '2025-10-15', 'Completed'),
(2, 'Programming Project', '2025-10-25', 'Pending'),
(3, 'Essay Submission', '2025-10-22', 'Pending'),
```

(4, 'Lab Report 2', '2025-10-24', 'Pending'),
(5, 'History Quiz', '2025-10-21', 'Completed');

Class Schedule Table:

```
CREATE TABLE Class_Schedule (  
    ScheduleID INT AUTO_INCREMENT PRIMARY KEY,  
    ClassID INT NOT NULL UNIQUE,  
    Class_Start_Time TIME NOT NULL,  
    Class_End_Time TIME NOT NULL,  
    Class_Days VARCHAR(50),  
    Classroom VARCHAR(50),  
    FOREIGN KEY (ClassID) REFERENCES Class(ClassID)  
    ON UPDATE CASCADE  
    ON DELETE CASCADE  
);  
  
insert dummy data class schedule:  
INSERT INTO Class_Schedule (ClassID, Class_Start_Time, Class_End_Time,  
Class_Days, Classroom)  
  
VALUES  
  
(1, '09:00:00', '10:30:00', 'Mon, Wed, Fri', 'Room 201'),  
(2, '11:00:00', '12:30:00', 'Tue, Thu', 'Room 305'),  
(3, '10:00:00', '11:30:00', 'Mon, Wed', 'Room 102'),  
(4, '13:00:00', '14:30:00', 'Tue, Thu', 'Lab 3'),  
(5, '08:30:00', '10:00:00', 'Fri', 'Room 108');
```

Study plan Table:

```
CREATE TABLE Study_Plan (  
    StudyPlanID INT AUTO_INCREMENT PRIMARY KEY,
```

```
StudentID INT NOT NULL,  
Subject_Name VARCHAR(100) NOT NULL,  
Duration INT,  
Status VARCHAR(20) DEFAULT 'Ongoing',  
FOREIGN KEY (StudentID) REFERENCES Student(StudentID)  
ON UPDATE CASCADE  
ON DELETE CASCADE  
);
```

Insert Dummy data. Study Plan:

```
INSERT INTO Study_Plan (StudentID, Subject_Name, Duration, Status)  
VALUES  
(1001, 'Algebra Review', 90, 'Ongoing'),  
(1001, 'Programming Basics', 60, 'Ongoing'),  
(1002, 'Literature Revision', 45, 'Completed'),  
(1003, 'Physics Problem Solving', 120, 'Ongoing'),  
(1004, 'World History Notes', 75, 'Pending'),  
(1005, 'Science Lab Prep', 60, 'Ongoing');
```

Reminder table:

```
CREATE TABLE Reminder (  
ReminderID INT AUTO_INCREMENT PRIMARY KEY,  
StudentID INT,  
TeacherID INT,  
DeadlineID INT,  
StudyPlanID INT,  
ScheduleID INT,
```

```
Message VARCHAR(255) NOT NULL,  
Reminder_Time DATETIME NOT NULL,  
Notification_Type VARCHAR(50),  
FOREIGN KEY (StudentID) REFERENCES Student(StudentID)  
ON UPDATE CASCADE  
ON DELETE SET NULL,  
FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID)  
ON UPDATE CASCADE  
ON DELETE SET NULL,  
FOREIGN KEY (DeadlineID) REFERENCES Deadline(DeadlineID)  
ON UPDATE CASCADE  
ON DELETE SET NULL,  
FOREIGN KEY (StudyPlanID) REFERENCES Study_Plan(StudyPlanID)  
ON UPDATE CASCADE  
ON DELETE SET NULL,  
FOREIGN KEY (ScheduleID) REFERENCES Class_Schedule(ScheduleID)  
ON UPDATE CASCADE  
ON DELETE SET NULL  
);
```

Insert dummy data Reminder Table:

```
INSERT INTO Reminder (StudentID, TeacherID, DeadlineID, StudyPlanID,  
ScheduleID, Message, Reminder_Time, Notification_Type)  
VALUES  
  
(1001, 2001, 1, 1, 1, 'Study for Algebra Test', '2025-10-19 18:00:00', 'Email'),  
(1001, 2002, 3, 2, 2, 'Complete Programming Project', '2025-10-24 20:00:00', 'Push'),
```

(1002, 2003, 4, 3, 3, 'Submit Literature Essay', '2025-10-21 08:00:00', 'Email'),
(1003, 2004, 5, 4, 4, 'Finish Physics Lab Report', '2025-10-23 17:00:00', 'SMS'),
(1004, 2005, 6, 5, 5, 'Prepare for History Quiz', '2025-10-20 19:00:00', 'Email'),
(1005, 2004, 5, 6, 4, 'Review Science Notes', '2025-10-22 09:00:00', 'Push');

6.0 SQL Queries

The following section details the ten SQL queries developed to demonstrate the database's functionality. These queries range from simple data retrieval to complex multi-table joins and analytical operations, showcasing the system's ability to support real-world academic management tasks.

Simple Queries

Query 1: View All Students

```
SELECT * FROM Student;
```

This query performs a basic selection of all student records, displaying their ID, name, and email. It serves as a foundational data retrieval operation, providing a complete directory of students within the system.

Query 2: Find Pending Deadlines

```
SELECT * FROM Deadline  
WHERE Status = 'Pending'  
ORDER BY Due_Date ASC;
```

This query filters the Deadline table to list all assignments with a 'Pending' status, ordered by their due date. It demonstrates data filtering and sorting, offering students and teachers a clear view of upcoming tasks in order of urgency.

Intermediate Queries

Query 3: Classes with Teacher Information

```
SELECT c.ClassID, c.Class_Name, CONCAT(t.Teach_FName, ' ', t.Teach_LName) AS  
Teacher_Name, t.Department  
FROM Class c  
INNER JOIN Teacher t ON c.TeacherID = t.TeacherID;
```

By joining the Class and Teacher tables, this query provides a clear overview of which teacher is responsible for each class, including their department. It demonstrates a fundamental INNER JOIN operation to combine related data from two tables.

Query 4: Student Enrollment Count

```

SELECT
    s.StudentID,
    CONCAT(s.Stud_FName, ' ', s.Stud_LName) AS Student_Name,
    COUNT(sc.ClassID) AS Classes_Enrolled
FROM Student s
LEFT JOIN Student_Class sc ON s.StudentID = sc.StudentID
GROUP BY s.StudentID, s.Stud_FName, s.Stud_LName
ORDER BY Classes_Enrolled DESC;

```

This query counts the number of classes each student is enrolled in using a LEFT JOIN and GROUP BY. It is useful for identifying student courses and includes those with zero enrollments, demonstrating basic data aggregation.

Query 5: Students Enrolled in Computer Science Classes

```

SELECT
    s.StudentID,
    CONCAT(s.Stud_FName, ' ', s.Stud_LName) AS Student_Name,
    s.Stud_Email
FROM Student s
WHERE s.StudentID IN (
    SELECT sc.StudentID
    FROM Student_Class sc
    INNER JOIN Class c ON sc.ClassID = c.ClassID
    INNER JOIN Teacher t ON c.TeacherID = t.TeacherID
    WHERE t.Department = 'Computer Science'
);

```

Utilizing a subquery, this query identifies all students who are enrolled in any class offered by the Computer Science department. It showcases the use of a subquery to filter results based on a condition that involves multiple related tables.

Complex Queries

Query 6: Teacher Workload Analysis

```

SELECT
    t.TeacherID,
    CONCAT(t.Teach_FName, ' ', t.Teach_LName) AS Teacher_Name,
    t.Department,
    COUNT(DISTINCT c.ClassID) AS Classes_Teaching,
    COUNT(DISTINCT sc.StudentID) AS Total_Students,
    COUNT(d.DeadlineID) AS Total_Deadlines
FROM Teacher t
LEFT JOIN Class c ON t.TeacherID = c.TeacherID
LEFT JOIN Student_Class sc ON c.ClassID = sc.ClassID

```

```

LEFT JOIN Deadline d ON c.ClassID = d.ClassID
GROUP BY t.TeacherID, t.Teach_FName, t.Teach_LName, t.Department
ORDER BY Total_Students DESC;

```

This complex query generates a workload report for each teacher by counting their distinct classes, total students, and assigned deadlines. It uses multiple LEFT JOINS and aggregate functions with DISTINCT to provide valuable administrative insights.

Query 7: Upcoming Deadlines (Next 7 Days)

```

SELECT
    d.DeadlineID,
    d.Title AS Assignment,
    c.Class_Name,
    CONCAT(t.Teach_FName, ' ', t.Teach_LName) AS Teacher,
    d.Due_Date,
    DATEDIFF(d.Due_Date, CURDATE()) AS Days_Until_Due
FROM Deadline d
INNER JOIN Class c ON d.ClassID = c.ClassID
INNER JOIN Teacher t ON c.TeacherID = t.TeacherID
WHERE d.Status = 'Pending'
    AND d.Due_Date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL
14 DAY)
ORDER BY d.Due_Date ASC;

```

Designed for proactive planning, this query lists all pending deadlines due within the next week. It uses date functions to calculate the remaining days and joins multiple tables to provide context, demonstrating time-based data filtering.

Query 8: Student Performance Dashboard

```

SELECT
    s.StudentID,
    CONCAT(s.Stud_FName, ' ', s.Stud_LName) AS Student_Name,
    COUNT(DISTINCT sc.ClassID) AS Total_Classes,
    COUNT(d.DeadlineID) AS Total_Assignments,
    SUM(CASE WHEN d.Status = 'Completed' THEN 1 ELSE 0 END) AS
Completed_Assignments,
    SUM(CASE WHEN d.Status = 'Pending' THEN 1 ELSE 0 END) AS Pending_Assignments,
    ROUND(
        (SUM(CASE WHEN d.Status = 'Completed' THEN 1 ELSE 0 END) * 100.0) /
        NULLIF(COUNT(d.DeadlineID), 0),
        2
    ) AS Completion_Rate
FROM Student s
LEFT JOIN Student_Class sc ON s.StudentID = sc.StudentID

```

```

LEFT JOIN Class c ON sc.ClassID = c.ClassID
LEFT JOIN Deadline d ON c.ClassID = d.ClassID
GROUP BY s.StudentID, s.Stud_FName, s.Stud_LName
ORDER BY Completion_Rate DESC;

```

This query creates a comprehensive performance dashboard by calculating the total and completed assignments per student, resulting in a completion rate percentage. It highlights the use of conditional aggregation with CASE statements for advanced data analysis.

Query 9: Students with Study Plans and Pending Deadlines

```

SELECT
    s.StudentID,
    CONCAT(s.Stud_FName, ' ', s.Stud_LName) AS Student_Name,
    sp.Subject_Name,
    sp.Duration,
    r.Message AS Reminder_Message,
    r.Reminder_Time
FROM Student s
INNER JOIN Study_Plan sp ON s.StudentID = sp.StudentID
INNER JOIN Reminder r ON s.StudentID = r.StudentID
WHERE sp.Status = 'Ongoing'
    AND r.Reminder_Time >= CURDATE()
ORDER BY r.Reminder_Time ASC;

```

This advanced query identifies students who have an ongoing study plan for a subject and also have a pressing, reminded deadline in that subject. It correlates data across five tables using INNER JOINS and a subquery with EXISTS, showcasing deep relational integration.

Query 10: Class Schedule Analysis

```

SELECT
    cs.Classroom,
    cs.Class_Days,
    c.Class_Name,
    CONCAT(t.Teach_FName, ' ', t.Teach_LName) AS Teacher_Name,
    COUNT(sc.StudentID) AS Students_Enrolled,
    TIME_FORMAT(cs.Class_Start_Time, '%H:%i') AS Start_Time,
    TIME_FORMAT(cs.Class_End_Time, '%H:%i') AS End_Time
FROM Class_Schedule cs
INNER JOIN Class c ON cs.ClassID = c.ClassID
INNER JOIN Teacher t ON c.TeacherID = t.TeacherID
INNER JOIN Student_Class sc ON c.ClassID = sc.ClassID
GROUP BY cs.Classroom, cs.Class_Days, c.Class_Name, t.Teach_FName, t.Teach_LName,

```



```
cs.Class_Start_Time, cs.Class_End_Time  
ORDER BY cs.Classroom, cs.Class_Start_Time;
```

This query analyzes resource utilization by grouping classes by their scheduled day and classroom. It shows the number of classes, total students affected, and the time range of classes in each room, demonstrating advanced grouping and string aggregation with GROUP_CONCAT.

Example Output:

7.0 Application Script (Python/Java)

The primary purpose of my Python script is to serve as a comprehensive tool for Database Initialization and Validation. It automates the process of setting up the database, ensuring that it is created correctly and functions as expected. The workflow of the script is as follows:

1. **Database Connection & Creation:** The script first connects to a MySQL server. I specify an existing database name, and if the database doesn't exist, it must be created manually in MySQL beforehand.
2. **Schema Creation:** Using the Entity-Relationship Diagram (ERD) and the business scenario as a guide, the script runs SQL CREATE TABLE statements to construct the database structure. This includes defining all tables, primary keys, and foreign key relationships.
3. **Data Population:** After creating the tables, the script populates them with "dummy data." This both validates that the table structures can store realistic data and provides a dataset for testing all queries.
4. **Query Execution & Validation:** Finally, the script executes the 10 queries from Section 6.0 of this report. This step confirms that the database is correctly integrated with the Python application and can return accurate results.

Overall, this script demonstrates a complete workflow from an empty database to a fully functional system capable of handling complex queries and answering business questions efficiently.

Code snippet

```
import MySQLdb  
from datetime import datetime, date, time
```

```
#Database Connection information
```

```
Host = "localhost"
```

```
User = "root"
```

```

Passwd = "Your_password"
DB = "dummytable" #Database to connect. Please create an empty one and put it here

# Connecting to the database
def connect_to_database():
    try:
        db = MySQLdb.connect(host=Host,
                               user=User,
                               passwd=Passwd,
                               db=DB)
        print("Database connection Successful!")
        return db

    except MySQLdb.InterfaceError as e:
        print(f'Failed to connect to the database. please check if database exists: {e}')
        return None

#Main method
try:
    print("Assignment preparation")
    db = connect_to_database()
    if not db:
        print("Haven't connected to the database.")
    else:
        # Creating the cursor object to write queries
        cur = db.cursor()
        if create_tables(cur, db):
            insert_data(cur, db)

```

This Python code snippet establishes a connection to a MySQL database using the MySQLdb module. It first defines the connection parameters—host, user, password, and database name—then defines the `connect_to_database()` function, which attempts to connect and returns the connection object if successful, or `None` with an error message if it fails. In the main block, the script calls this function and checks whether the connection succeeded. If it does, it creates a cursor object to execute SQL queries, then calls two functions: `create_tables()` to build all required tables according to the database schema and `insert_data()` to populate them with sample data. The design ensures that SQL operations are only performed on a valid database connection and handles errors gracefully, providing a robust and integrated workflow for initializing and testing the database.

The script was tested using a rigorous, multi-stage validation process to ensure reliability and correctness:

1. **Incremental Development & Syntax Checking:** The script was written and tested in stages. First, the connection function was validated. Then, CREATE TABLE statements were tested one by one to catch any SQL syntax errors or foreign key constraint issues early.
2. **Data Integrity Checks:** After the insert_data() function ran, manual SELECT * FROM table; queries were executed via the Command line to verify that all dummy data was inserted correctly and that relational integrity (e.g., foreign keys) was maintained.
3. **Query Validation (Core Test):** The most critical test was the execution of the 10 queries from **Section 6.0**. For each query:
 - The output was visually inspected to ensure it made logical sense.
 - The results were cross-checked against the known dummy data to verify accuracy. For example, making sure that all information was inputted into the right columns of the tables database.

8.0 Testing and Validation

For testing, we took a comprehensive approach to ensure the database and all queries were fully functional and reliable. We began by validating the connection to MySQL, handling invalid credentials or missing databases gracefully, and confirming that all tables were created correctly with proper constraints. We then populated the tables with sample data and ran all eight queries, checking that the results were accurate. Simple queries, like listing all students or showing pending deadlines, worked as expected. More advanced queries, using JOINS, subqueries, and aggregation, such as teacher workload analysis and the student performance dashboard, processed relational data correctly and returned accurate metrics.

Testing Methodology

- **Database Connection:** Verified successful connections, handled invalid credentials or missing databases, and ensured proper closure.
- **Table Creation:** Checked for existing tables, validated foreign key relationships, and tested cascading updates and deletes.
- **Data Integrity:** Tested INSERT operations with the sample dataset, enforced UNIQUE and primary key constraints, and confirmed referential integrity.

- **Query Functionality:** Executed all queries, verified outputs for correctness and formatting, and tested edge cases and boundary conditions.

Test Results

- **Connection & Setup:** Database connection, Table creation, Data insertion, Foreign key enforcement was all successful and ran perfectly
- **Simple Queries:** Returned correct student lists and pending deadlines
- **Intermediate Queries:** Displayed classes with teachers, enrollment counts, and department student lists correctly.
- **Complex Queries:** Generated teacher workload metrics, filtered upcoming deadlines, calculated performance dashboard metrics, and aggregated schedule data accurately.

Error Handling

Implemented robust mechanisms to catch connection failures (`MySQLdb.InterfaceError`), prevent duplicate table creation, handle SQL syntax errors, maintain transaction integrity, and validate data types and constraints during insertion.

Limitations

The system requires a pre-existing MySQL database, uses a static sample dataset, assumes ideal conditions for some complex queries, and stores passwords in plaintext without advanced security features.

Validation Summary

Testing confirmed that all database operations execute successfully, queries return accurate results, referential integrity is maintained, and Python-MySQL integration supports both simple and complex operations. Overall, the system is fully functional for academic management tasks.

9.0 Reflections and Discussion

9.1 Design Choices and Justification

The design of the school scheduling database was guided by several key principles aimed at ensuring data integrity, minimizing redundancy, and supporting efficient query operations.

Normalization to Third Normal Form (3NF): The decision to normalize the database to 3NF was fundamental to eliminating data redundancy and ensuring

consistency [1]. By decomposing the initial CLASS DETAILS table through successive normalization stages, we eliminated partial and transitive dependencies, resulting in separate STUDENT, TEACHER, CLASS, and ENROLMENT tables. This approach prevents update anomalies and ensures that each piece of information is stored in only one location, reducing the risk of inconsistent data [2].

Many-to-Many Relationship Decomposition: The ENROLMENT table was introduced as a junction table to resolve the many-to-many relationship between STUDENT and CLASS entities [3]. This design choice allows students to enroll in multiple classes while enabling classes to accommodate multiple students, accurately reflecting real-world academic scenarios. The composite primary key (StudentID, ClassID) ensures that each student-class pairing is unique and prevents duplicate enrollments.

Foreign Key Constraints with Cascading Actions: We implemented foreign key constraints with ON UPDATE CASCADE and ON DELETE CASCADE actions throughout the schema [4]. This decision automates referential integrity maintenance. When a teacher is removed from the system, all associated classes are automatically deleted, preventing orphaned records. While this provides convenience, we recognized the trade-off: cascading deletes could inadvertently remove valuable historical data. In a production environment, soft deletes or archiving mechanisms might be more appropriate.

Weak Entity Design for Context-Dependent Tables: Entities such as REMINDER, DEADLINE, and CLASS SCHEDULE were designed as weak entities because their existence depends on parent entities like STUDENT, CLASS, and TEACHER [5]. For instance, a DEADLINE cannot exist without an associated CLASS, and a REMINDER loses meaning without a linked STUDENT or TEACHER. This dependency structure reinforces the logical relationships within the academic domain.

Use of AUTO_INCREMENT for Primary Keys: We employed AUTO_INCREMENT for primary keys in most tables (ClassID, DeadlineID, ReminderID, etc.) to simplify data insertion and ensure uniqueness without manual intervention [6]. This approach reduces the likelihood of primary key conflicts and streamlines the data entry process, particularly when multiple users interact with the system simultaneously.

9.2 Challenges Encountered

The development process presented several technical and conceptual challenges that required systematic problem-solving and iterative refinement.

Data Type Mismatches Between Python and MySQL: One of the most persistent issues involved incompatibilities between Python's datetime objects and MySQL's DATE, TIME, and DATETIME types [7]. Python's datetime.date objects needed explicit conversion to string format ('YYYY-MM-DD') before insertion into MySQL DATE columns. Similarly, time values required formatting as 'HH:MM:SS' strings. We addressed this by implementing

This challenge highlighted the importance of understanding data type mappings across different programming environments and the need for robust data validation layers.

Debugging Complex SQL Queries: The intermediate and complex queries, particularly those involving multiple INNER JOINS, LEFT JOINS, and subqueries, frequently returned unexpected results or empty result sets during initial testing [8]. For example, Query 9 (Students with Study Plans and Pending Deadlines) initially failed to return any rows because the subquery logic incorrectly filtered out all matches. We resolved this by:

1. Breaking down complex queries into smaller components and testing each JOIN independently
2. Using EXPLAIN statements to analyze query execution plans and identify performance bottlenecks
3. Verifying that test data satisfied all join conditions before expecting results
4. Adding comprehensive WHERE clause logging to trace filtering logic

This iterative debugging process reinforced the importance of modular query construction and systematic testing [9].

Naming Inconsistencies Between Schema Definition and Data Insertion: A critical error emerged from a discrepancy between table naming conventions: the CREATE TABLE statement defined a table named Enrollment, but subsequent INSERT statements attempted to populate a table called Student_Class [10]. This mismatch caused runtime errors that were initially difficult to diagnose because the Python script did not provide clear feedback about non-existent tables.

We resolved this issue by:

- Standardizing all table names in both schema creation and data manipulation scripts
- Implementing a centralized configuration file that defined table names as constants

- Adding validation checks to verify table existence before executing INSERT operations

This experience underscored the necessity of maintaining consistent naming conventions throughout the development lifecycle and the value of using configuration management tools to prevent such discrepancies.

Foreign Key Constraint Violations During Data Insertion: When populating tables with dummy data, we encountered foreign key constraint violations because child records were inserted before their referenced parent records existed [11]. For instance, attempting to insert a CLASS record with TeacherID = 2001 before inserting the corresponding TEACHER record caused the operation to fail. We addressed this by carefully ordering INSERT statements to respect dependency hierarchies: STUDENT and TEACHER first, followed by CLASS, then ENROLMENT, DEADLINE, STUDY_PLAN, CLASS_SCHEDULE, and finally REMINDER.

9.3 Team Collaboration Challenges and Solutions

Working as a team of six members presented unique challenges that required deliberate coordination strategies and clear communication protocols.

Task Distribution and Workload Balance: Initially, distributing tasks equitably among six team members proved difficult. Some members had stronger SQL skills while others were more proficient in Python programming. We addressed this by conducting a skills assessment meeting where each member identified their strengths and areas for development [12]. Based on this assessment, we divided the project into parallel workstreams:

- Two members focused on ERD design and normalization
- Two members handled database implementation and table creation
- Two members developed SQL queries and the Python application script

This distribution ensured that everyone contributed meaningfully while also learning from teammates with complementary skills.

Version Control Conflicts: With six people working on different components simultaneously, we initially experienced frequent merge conflicts in our shared code repository [13]. Team members would overwrite each other's changes, particularly in the main Python script file. We resolved this by:

- Adopting a strict Git branching strategy where each member worked on feature branches

- Implementing mandatory code reviews before merging any changes to the main branch
- Scheduling daily stand-up meetings to discuss who was working on which files
- Using clear commit messages that described the changes made

These practices significantly reduced conflicts and improved code quality through peer review.

Communication Gaps and Misunderstandings: The naming inconsistency issue (Enrollment vs. Student_Class) stemmed from inadequate communication between the team members responsible for schema design and those implementing data insertion [14]. To prevent similar issues, we established:

- A shared Google Document containing all naming conventions and design decisions
- Weekly synchronization meetings where all six members reviewed progress and design choices
- A designated team lead who ensured consistency across all deliverables
- A standardized code commenting convention that explained the purpose of each function

Varying Levels of Database Knowledge: Team members had different levels of familiarity with database concepts, particularly normalization and complex SQL queries. Some members struggled with understanding foreign key relationships and JOIN operations [15]. We tackled this through:

- Peer teaching sessions where more experienced members explained concepts to others
- Creating a shared repository of learning resources and tutorials
- Pair programming sessions for complex tasks like writing multi-table queries
- Encouraging questions in group discussions without judgment

This collaborative learning approach strengthened the entire team's technical capabilities.

Meeting Scheduling and Time Zone Differences: Although all team members were local, coordinating six busy schedules proved challenging. We addressed this by:

- Using Doodle polls to find common meeting times
- Recording key meetings for members who could not attend
- Using asynchronous communication tools like Slack for quick updates
- Establishing core collaboration hours (Tuesday and Thursday evenings) when everyone was available

Quality Assurance and Testing Responsibility: Initially, no one took ownership of comprehensive testing, leading to bugs being discovered late in the development cycle. We resolved this by:

- Assigning one member as the quality assurance lead
- Implementing a testing checklist that covered all functionalities
- Requiring every team member to test their own code before submitting it for review
- Conducting group testing sessions where we collectively ran all queries and verified outputs

Integration Challenges: Combining work from six different contributors into a cohesive final product required careful integration planning [16]. Different coding styles, inconsistent formatting, and varying documentation standards created integration difficulties. We standardized our approach by:

- Adopting PEP 8 style guidelines for Python code
- Using consistent SQL formatting (uppercase keywords, specific indentation)
- Creating a master template for documentation that everyone followed
- Conducting integration sprints where we combined components incrementally rather than all at once

These collaborative challenges taught us that technical skills alone are insufficient for successful project completion. Effective teamwork requires clear communication protocols, established workflows, mutual respect, and a willingness to support each other's learning.

9.4 Lessons Learned

This project provided valuable insights into database design, implementation, and integration that extend beyond technical proficiency.

Importance of Comprehensive Planning: The normalization process demonstrated that investing time in upfront design significantly reduces complexity

during implementation [17]. By thoroughly analyzing functional dependencies and systematically decomposing tables, we avoided the need for costly schema modifications later. Future projects would benefit from even more detailed entity-relationship modeling before any code is written.

Value of Incremental Development and Testing: The staged approach to script development (testing database connection, then table creation, then data insertion, and finally query execution) proved invaluable [18]. This methodology allowed us to isolate and resolve errors at each stage rather than confronting a cascade of failures at the end. The principle of "test early, test often" became a cornerstone of our development practice.

Need for Robust Error Handling: The Python script's error handling mechanisms, particularly for connection failures and SQL syntax errors, prevented silent failures that could have corrupted data or left the database in an inconsistent state [19]. We learned to anticipate failure modes and implement try-except blocks with meaningful error messages, which greatly accelerated debugging.

Documentation as a Development Tool: Maintaining detailed documentation throughout the project (from business rules to entity descriptions to query explanations) served not only as a deliverable but as a critical reference during development [20]. When debugging complex queries, referring back to the business rules helped us verify that our SQL logic correctly implemented the intended relationships.

Understanding the Trade-offs in Design Decisions: Every design choice involved trade-offs. For example, while cascading deletes simplified data management, they also introduced risks of unintended data loss. Similarly, using `AUTO_INCREMENT` primary keys simplified insertion but made manual data migration more complex. Recognizing these trade-offs and making informed decisions based on the specific use case is a crucial skill in database engineering [21].

Integration Complexity Between Programming Languages and Databases: The challenges with data type conversions and query parameterization highlighted that database systems do not exist in isolation [22]. Understanding how Python interacts with MySQL (including connection pooling, cursor management, and result set handling) is essential for building robust applications. This project reinforced that effective database work requires proficiency in both SQL and at least one programming language.

Team Dynamics as a Success Factor: Perhaps the most significant lesson was recognizing that team dynamics and collaboration skills are as important as

technical expertise. Our success depended not just on individual contributions but on our ability to communicate effectively, resolve conflicts constructively, and support each other's learning. The challenges we faced in coordinating six team members taught us project management, leadership, and interpersonal skills that will be valuable throughout our careers [23].

Adaptability and Problem-Solving: The various technical and collaboration challenges required us to be flexible and creative in finding solutions. Whether it was debugging complex queries, resolving merge conflicts, or finding meeting times that worked for everyone, we learned to approach problems systematically and remain patient when solutions were not immediately obvious [24].

10.0 Conclusion

The development of the Orama Scheduler successfully demonstrated how web-based automation and AI integration can improve the way students manage their academic schedules. The project achieved its main objectives by creating a responsive, user-friendly system capable of generating and managing personalized study plans, integrating an intelligent chatbot for real-time assistance, and laying the groundwork for secure database storage using MySQL. Key outcomes include the successful implementation of an AI-driven chatbot through the OpenAI API, seamless communication between the frontend and backend using Node.js and TypeScript, and a well-structured design that supports future database and API extensions.

Beyond technical achievement, the project provided significant educational and practical value. It strengthened our understanding of full-stack development, database design, and API integration while emphasizing teamwork, problem-solving, and debugging in real-world scenarios. In application, the Orama Scheduler has the potential to enhance student productivity by simplifying time management and promoting effective study habits. With future integration of calendar synchronization, reminders, and performance analytics, the system could evolve into a comprehensive academic planning platform that supports both students and educators in managing their time more efficiently.

[link to GitHub repository](#)

11.0 Team Members' Contribution

S/N	Name and Surname	Student Number	Key Contribution

1	SIYABULELA MUZI MALINGA	4370961	Conclusion section, SQL queries development and testing, database validation
2	CHARITY NDUVHO MALADA	4338756	Reflections and Discussion section, business rules extraction and documentation, initial brainstorming and scenario development
3	NOMPUMELELO NTHULANE	4323787	Introduction and Background section, SQL queries development and implementation, query documentation
4	BLESSING AGGREY BANDA	4340659	Python script development and testing, database implementation coding, GitHub repository setup and management, application integration
5	SIBABALWE NOMVETE	4352145	Entity-Relationship Diagram (ERD) design using Draw.io, normalization documentation, relational schema mapping
6	SIBUSISIWE ZULU	4270237	Entity-Relationship Diagram (ERD) design using Draw.io, database table creation and constraints, data insertion and validation

12.0 References

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [2] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.
- [3] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Harlow, UK: Pearson, 2015.
- [4] "MySQL 8.0 Reference Manual: FOREIGN KEY Constraints," Oracle Corporation, 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>. [Accessed: Oct. 10, 2025].

- [5] P. Rob and C. Coronel, *Database Systems: Design, Implementation, and Management*, 13th ed. Boston, MA, USA: Cengage Learning, 2019.
- [6] "MySQL 8.0 Reference Manual: AUTO_INCREMENT Handling in InnoDB," Oracle Corporation, 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-auto-increment-handling.html>. [Accessed: Oct. 10, 2025].
- [7] "MySQLdb User's Guide," SourceForge, 2024. [Online]. Available: <https://mysqlclient.readthedocs.io/>. [Accessed: Oct. 11, 2025].
- [8] B. Forta, *SQL in 10 Minutes a Day, Sams Teach Yourself*, 5th ed. Indianapolis, IN, USA: Sams Publishing, 2019.
- [9] "20 Advanced SQL Techniques with Practical Examples," Medium, 2023. [Online]. Available: <https://medium.com/@techsuneel99/20-advanced-sql-techniques-with-practical-examples-b47490d9896d>. [Accessed: Oct. 11, 2025].
- [10] "How do I connect to a MySQL Database in Python?" Stack Overflow, 2024. [Online]. Available: <https://stackoverflow.com/questions/372885/how-do-i-connect-to-a-mysql-database-in-python>. [Accessed: Oct. 11, 2025].
- [11] A. Beaulieu, *Learning SQL: Generate, Manipulate, and Retrieve Data*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [12] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, 2020. [Online]. Available: <https://scrumguides.org/>. [Accessed: Oct. 11, 2025].
- [13] S. Chacon and B. Straub, *Pro Git*, 2nd ed. New York, NY, USA: Apress, 2014.
- [14] F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*, Anniversary ed. Boston, MA, USA: Addison-Wesley, 1995.
- [15] B. W. Tuckman, "Developmental sequence in small groups," *Psychological Bulletin*, vol. 63, no. 6, pp. 384-399, 1965.
- [16] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008.
- [17] G. Simsion and G. Witt, *Data Modeling Essentials*, 3rd ed. San Francisco, CA, USA: Morgan Kaufmann, 2005.
- [18] M. J. Hernandez, *Database Design for Mere Mortals: A Hands-On Guide to Relational Database Design*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2013.
- [19] "Python Error Handling Best Practices," Real Python, 2024. [Online]. Available: <https://realpython.com/python-exceptions/>. [Accessed: Oct. 11, 2025].

[20] D. C. Hay, *Data Model Patterns: Conventions of Thought*. New York, NY, USA: Dorset House Publishing, 1996.

[21] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[22] "Complex SQL Queries," PopSQL Blog, 2024. [Online]. Available: <https://popsql.com/blog/complex-sql-queries>. [Accessed: Oct. 11, 2025].

[23] P. Lencioni, *The Five Dysfunctions of a Team: A Leadership Fable*. San Francisco, CA, USA: Jossey-Bass, 2002.

[24] D. A. Kolb, *Experiential Learning: Experience as the Source of Learning and Development*, 2nd ed. Upper Saddle River, NJ, USA: Pearson FT Press, 2014.

13.0 Plagiarism Declaration

We are aware and understand that plagiarism is wrong and punishable according to the UWC policy. I confirm that this work is our own and that we have not plagiarized our work nor allowed anyone to copy our report or code.